Green Is Not Done: Nine Silent Failures on the Path to a Complete OSM Extraction, and the Capacity Recovered by Multi-Architecture Images

Green Is Not Done: Nine Silent Failures on the Path to a Complete OSM Extraction, and the Capacity Recovered by Multi-Architecture Images

Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: the 2026-08-29 → 08-31 campaign to complete the US sub-national tiers of the self-hosted planet split — 25 commits across the `facetwork` runtime and the `fwh_osm` domain — together with the fleet's execution records, MinIO bucket inventories, and benchmark measurements taken on the fleet's own hosts. This paper continues Become Your Own Geofabrik, which describes building the substrate; here we describe finishing the job.

Abstract

A pipeline that *works* and a pipeline that *completes* are different achievements, and the distance between them is not made of bugs in the happy path. We report on a three-day campaign to extract the remaining sub-national tiers of a self-hosted OpenStreetMap planet split — 51 US states and 3,167 counties that had sat 36 days stale while the continent tier above them was same-day fresh. The workflow had been written, reviewed, and deployed. It nonetheless failed **six consecutive times**, and in four of those runs it reported success while producing nothing.

The nine defects we found share one shape, and it is not the shape a test suite looks for: **a second code path — almost always a recovery path — omits a guarantee that the primary path carefully establishes.** A stuck-step sweep recreated tasks without their resource floors. Three of four call sites of a blocking primitive lacked the heartbeat the fourth had. A merge list was replaced rather than appended, silently disabling a refresh knob across nine configurations. A publish step, unscoped, re-uploaded 95 GB of continent extracts it had never built and overwrote canonical data. In each case the primary path kept working, so nothing failed — until recovery was needed, at which point the system quietly did the wrong thing and returned zero.

We then report a second, independent result from the same campaign. The fleet's container image had been built for a single architecture, forcing two of four hosts to execute every task under CPU emulation. Making the image multi-architecture required **no code changes and four minutes of build time**, and returned, by direct measurement on the same machine: **32× on interpreter startup, 14× on bulk hashing, 71× on a tight interpreter loop, and 35× on container start-to-exit.** It restored 32 previously-unusable runner slots and converted one emulated host into a second heavy-tier worker that then carried 42% of a 3,167-way fan-out. We argue that architecture portability in a heterogeneous fleet is not a packaging preference but a capacity decision, and that its cost is routinely overestimated because the failure it prevents — silent emulation — presents as ordinary slowness rather than as a fault.

---

1. Introduction

The substrate paper (Become Your Own Geofabrik) ends where most infrastructure papers end: the mechanism exists, it has been exercised, the design decisions are recorded. This paper begins one tier lower, with the observation that the mechanism had not actually finished its job.

The self-hosted split maintains a Geofabrik-compatible tree: eight continent extracts, kept current by a nightly re-split, and beneath them roughly 3,900 country, state and county extracts. On 2026-08-29 the continents were same-day fresh and everything below them was a **July vintage, 36 days old**. Nothing was broken. No alarm had fired. The tier simply had no process that refreshed it, and the absence of a process is not an error condition.

That is the first observation of this paper and it recurs throughout: *the failure modes that survive longest are the ones that do not raise anything.* What follows is an account of nine such failures, encountered consecutively while trying to refresh 3,218 extracts, and of a tenth problem — architectural homogeneity assumed by an image build — that had been quietly costing the fleet half its hosts.

We make three claims:

1. **Recovery paths systematically lose the guarantees of primary paths**, and this is a structural consequence of how such paths are written (§3, §4).
2. **A no-op that reports success is worse than an error**, and several of our defects were undetectable precisely because they were well-behaved (§5).
3. **Multi-architecture images are a capacity intervention**, with a measured return far exceeding their build cost, and the reason they are deferred is that emulation degrades rather than fails (§6).

---

2. The task and its shape

The target was two tiers:

tier	count	source of region boundaries	workflow
US states	51	US Census TIGER shapefiles	`BuildPlanetExtracts(scope=subnational)`
US counties	3,167	OSM `admin_level=6` relations	`BuildAdminFanout`

These use *different keying rules*, which matters later. TIGER **constructs** the key `north-america/us/<state>`; the OSM-boundary path **derives** its prefix from the source region. A continent source can therefore only ever key children directly beneath that continent, so `north-america` @ `admin_level 4` yields `north-america/<state>` — a tier too shallow. A prior run had published 99 extracts at that wrong depth (20.09 GB), which had to be deleted before this campaign began.

The county tier is a 3,167-way fan-out over 51 parent extracts, bounded by a `foreach limit` (`width: 6`). Each child downloads its parent state extract from the object store (0.1–1.4 GB), filters `admin_level=6` boundaries, assembles polygons with `osmium export`, cuts one extract per county, and publishes.

---

3. The recurring defect shape

Before enumerating the failures, we state the pattern they share, because it is the paper's central claim and the individual instances are otherwise merely a list.

> **A guarantee established at one call site is absent at another call site of the same primitive — and the second site is usually the recovery path.**

Why recovery paths specifically? Three reinforcing reasons:

- **They are written later**, often during an incident, when the goal is to restore progress rather than to preserve invariants.
- **They have less context.** Our stuck-step sweep runs inside the runner service, which has no evaluator context; the primary path runs inside the evaluator, where facet definitions and the program AST are at hand. The primary path can *derive* a task's routing; the recovery path structurally cannot, so it constructed a bare task instead.
- **They are exercised rarely and never on the happy path**, so tests written against normal operation cannot see them. Every one of our defects had green tests.

The practical consequence is a search strategy, which we give in §7: **grep the primitive, not the handler.**

---

4. Nine failures

We present these in the order encountered, because the order is itself informative: each fix revealed the next defect, which had been masked.

4.1 A host path as a container default (`be5e5a2`)

`_PLANET_DIR` defaulted to `/Volumes/afl_data/osm-selfhost`. This is correct on a bare-metal runner — the replication publisher serves the tree from exactly there — and catastrophic inside a container, where the path does not exist and `os.makedirs` cheerfully creates it **in the container's writable layer**. An ~80 GB planet download therefore streamed into a 58 GB Docker VM overlay.

It filled, and MongoDB — which stored its data on the same VM filesystem — died:

```
WiredTiger error 28 (ENOSPC) → −31804 (WT_PANIC) → fassert → Restarting(133)
```

The fleet's database crash-looped for ~35 minutes. Meanwhile **3.6 TiB sat free on the external volume two directories away.**

Nothing errored at the point of the mistake. `mkdir` succeeded; the download proceeded normally. This is the canonical form of the defect class: the wrong behaviour is indistinguishable from the right one until a resource is exhausted.

The fix resolves rather than assumes: explicit `FW_PLANET_DIR` → the host path **if it actually exists** → the mounted scratch. Existence is the discriminator because that is precisely what distinguishes the two environments. A free-space guard was added behind it — not because the resolution is expected to fail, but because the failure it guards is **shared-fate**: filling a Docker VM disk does not fail one task, it kills every container storing data on that VM.

4.2 A cache check that could neither pass nor finish (`87b6902`)

`fetch_planet` decided "already present" by comparing the local file's md5 to the published `planet-latest.osm.pbf.md5`. For a **maintained** planet this is guaranteed false forever: `update_planet` advances our copy *past* upstream by applying replication diffs, so the checksums diverge permanently by design. The check would have re-downloaded 92 GB on every single run.

Worse, it could not complete. Measured on the fleet's storage, hashing the mounted planet runs at 27 MB/s, so a full pass takes **~57 minutes** — nearly double the 30-minute stuck-task timeout that then reclaimed the task mid-hash, which started *another* concurrent hash of the same file.

The replacement reads the PBF replication header: **0.46 s**, and it returns the artifact's *vintage* (`seq=5100, ts=2026-08-30`) rather than a boolean. The general lesson: **a checksum comparison against upstream is invalid for any artifact we deliberately mutate.** Freshness must be expressed in terms the artifact itself carries.

4.3–4.5 Three missing heartbeats (`87b6902`, `7909e94`, `55c14d2`)

`bootstrap_batched`, `_publish_tree` and `fetch_planet` are long blocking calls with no loop boundary at which to signal liveness. The codebase already had the mechanism — a ticker thread (`_heartbeating`) plus cooperative cancellation — and its docstring already documented the exact failure it prevents, citing an earlier incident in which two hosts each re-downloaded the same 40.5 GB extract.

It was applied to **one of four** call sites.

The consequences, measured:

handler	consequence
`DownloadPlanet` / `UpdatePlanet`	reclaimed every ~31 min; 5 reclaims → dead-letter after 2.6 h
`ExtractRegions`	reclaimed *while publishing*; **7 concurrent executions**, each restarting from batch 1, each running a ~12 GiB osmium pass → OOM (`SIGKILL`)
`PublishExtracts`	found by inspection before it failed

The `ExtractRegions` case deserves emphasis because it produced a misleading diagnosis. The visible symptom was an out-of-memory kill, which points at batch size. The actual cause was **duplicate executions manufactured by the recovery path**: the batch size was fine at 12.16 GiB of a 23.43 GiB VM; seven of them were not. We had already reduced `batch_size` twice before finding this, which is a reminder that *the visible resource symptom is not reliably the cause.*

Note also the progression in how the three were found: the first by a dead-letter after hours, the second by an OOM after hours, the third **by grepping the primitive** in seconds. That difference is the paper's practical recommendation.

4.6 Planet-scale I/O for a continent-scale job (`47b02e1`, `9f459170`)

US states intersect only North America. The workflow nonetheless cut them from the 92 GB planet, because `BuildPlanetExtracts` hardwired the planet as source — even though `tiger_fetch`'s own docstring described the cheaper path: "*the planet, or the north-america continent extract for a cheaper pass.*"

Adding a `source_region` parameter reduced the source from **92 GB to 21 GB**: ~4.4× less I/O, and proportionally less resident state per region. Since each batch is a full pass over the source, source size and batch size are one decision, not two.

The measured cost of `complete_ways` extraction is **~3–4 GB per region**, so `batch_size=25` implies 75–100 GB of resident state — which no VM on this fleet has. It OOM'd at both 14 GiB and 24 GiB before we reduced it to 4.

4.7 No resume (`abd7f38`)

`ExtractRegions` had no resume. Every reclaim restarted at batch 1 of 13, so a 30-minute reclaim window meant the run **never reached batch 4** — while 40 of 51 states sat complete on disk.

The skip invariant we adopted is "*an extract is valid if it is newer than the source it was cut from.*" This requires no threshold to tune and is self-correcting: the nightly re-split advances the continent extract, which makes every derived child stale automatically. An age threshold would have to be guessed and then kept in step with a cadence by hand.

One deliberate exception: **zero-length outputs are never skipped.** A killed `osmium` leaves truncated files behind — 25 were present on the host — and skipping one would let a broken extract survive every future run.

4.8 An unscoped publish (`5f4cc89`)

`PublishExtracts` globbed its entire output tree. With the default output directory that matched **59 files: the 51 US states and all 8 continent extracts** — ~95 GB the run had never built.

Three problems in ascending severity:

1. **Waste**: every run re-uploaded ~95 GB of unchanged continents.
2. **Failure**: `CompleteMultipartUpload` returned `InvalidPart` on the 38 GB europe object, because the task had been reclaimed mid-upload (§4.3–4.5) and the second uploader invalidated the first's session. The run burned 5.7 h and published nothing.
3. **Data-loss risk**: it *overwrote the canonical continent extracts* with whatever happened to sit on that host's disk. Three continents (africa, asia, central-america) were in fact overwritten before we caught it. They were harmless only because the local tree happened to be current — verified after the fact, all at replication sequence 5100.

The fix scopes the publish to the regions the run actually produced, and refuses zero-length files outright.

4.9 A merge list replaced instead of appended (`1e8366a8`, `32b9678`)

Our own earlier fix in this same campaign added a key to a `merge_defaults` list by **replacing** it. Nine configuration sets silently lost `refresh_after_days`, which defaults to `0` — meaning *never refresh*.

The result was the campaign's most instructive failure, because it looked perfect:

```
run completed | 54/54 tasks | retry=0 | 20 minutes
"admin_level=6 of north-america/us/texas: 227 extracts published"
```

Zero counties were built. Every one was skipped as already-published, and the bucket still held the 36-day-old vintage. The "227" came from the resume count being reported in the same field as the build count, making a total no-op indistinguishable from a full rebuild.

We changed the reporting to state both numbers and to name the responsible knob:

```
"NOTHING BUILT — all 227 region(s) skipped as already-current
 (refresh_after_days=0; 0 means never refresh)"
```

This is the same discipline the codebase had already adopted elsewhere, where polygon generation raises if it found candidates and kept none. The principle: **a job that produces nothing must say so loudly, because `rc=0` plus a plausible number is how a stale tier survives for 33 days unnoticed.**

4.10 The stuck-step sweep stripped capability routing (`ea768864`)

Found not by a failure but by a *statistics tool* built to answer an unrelated question. Attributing per-host work revealed that a 7.75 GiB host had claimed tasks carrying a `Requires(memory_gb=10)` floor.

The cause: the sweep recreates a task for a stuck step, but constructed it bare — no `requires`, `required_features`, `environment_hash`, `kind`, `timeout_ms`, not even `created`. The normal path derives all of these from the step's mixins and AST. **The recovery path discarded every routing guarantee at exactly the moment recovery made them matter.**

Seven tasks in one run carried `requires: {}`. They completed only because those states happened to be small. Dropping `required_features` is the same defect that had previously left two scheduled snapshots unclaimable for two days; dropping `environment_hash` would route a script task to a runner without its virtualenv.

The fix inherits routing from the newest prior task on the same step — which the primary path built correctly — and, where nothing is inheritable, **warns rather than silently emitting an unrouted task.** Measured on the triggering run, 9 of 11 swept tasks had an inheritable predecessor, so the warning path is real.

---

5. Why four of six failures were green

Four of the six failed runs reported success. This is worth isolating, because it is the property that made the campaign expensive rather than merely long.

what was reported	what happened
`54/54 tasks, retry=0, completed`	zero counties built
`227 extracts published`	227 skipped, 0 built
`manifest verified: amd64 arm64`	12 of 24 blobs had no data
`rc=0, 3h20m, 40 GB downloaded` (prior campaign)	0 extracts published

Each of these is a **well-formed report of a meaningless outcome**. None is a crash, an exception, or a non-zero exit. Conventional monitoring — alert on errors, alert on exit codes — is blind to all of them.

We draw two design rules:

R1. Report the quantity that would be zero on failure, not the quantity that happens to be available. "Published" conflated built and skipped; separating them made the no-op self-evident.

R2. Verify the property you depend on, not a proxy for it. Our own verification claimed `manifest verified: amd64 arm64` for an image that could not be pulled at all, because it checked that platforms were *listed* rather than that their layers had *data*. A `HEAD` request is answered from metadata; a one-byte ranged `GET` forces the registry to open the file. The corrected check costs bytes and catches the real condition.

R2 has a corollary we now enforce mechanically: **a value that cannot physically be true is a free assertion.** Our statistics tool reported "peak memory 21.6 GB" for a host whose entire VM is 7.75 GiB. That is impossible, and the impossibility — not a test — exposed a misattribution bug. The tool now flags any parsed peak exceeding a host's advertised capacity as suspected misattribution rather than usage.

---

6. Multi-architecture images as a capacity intervention

6.1 The situation

The fleet is four machines: two Apple Silicon (M2 Max, 34 GB) and two 2018 Intel Mac minis (i7-8700B, 6C/12T, 34 GB). The runner image was built `linux/arm64` only. The Intel hosts therefore executed **every task under `qemu-aarch64` user-mode emulation**.

The operational signature of this had been recorded but not diagnosed: *"load 17 on 12 cores, 30–60 minutes to start 16 runners*", and a recurring status display showing hosts as `MISSING` for half an hour that operators had learned to read as "emulation, not a fault". The fleet had, in effect, been running at half its host count for months, and the cost was invisible because **emulation degrades rather than fails.**

6.2 Cost of the change

Essentially nil. A trial `linux/amd64` build completed in **4 minutes cold, zero cached steps, with no source changes**:

- the base image (`python:3.12-slim`) is already multi-arch;
- `apt` packages resolve per-architecture automatically;
- `tippecanoe` is compiled from source, so architecture-agnostic by construction;
- the `pmtiles` download already had a `TARGETARCH` case;
- GraphHopper is a JVM jar;
- `osmium`, `shapely`, `pymongo` all publish wheels for both architectures.

The rollout script already had a `FW_FLEET_PLATFORM` variable. Making the fleet architecture-independent was, in the end, a one-line default change plus removal of a `DOCKER_DEFAULT_PLATFORM=linux/arm64` pin from two host-local wrappers — a pin which, left in place, would have *forced* the x86 hosts onto the emulated half of the new image.

6.3 Measured benefit

Controlled A/B on one machine (server1, i7-8700B), same image content, differing only in architecture:

metric	native amd64	emulated arm64	gain
Python imports (osmium, shapely, pymongo, boto3)	**0.57 s**	18.21 s	**32×**
sha256 over 75 MB	**0.20 s**	2.76 s	**14×**
interpreter loop (3M iterations)	**0.65 s**	45.95 s	**71×**
container start → exit	**2 s**	70 s	**35×**

The 71× on the interpreter loop is the extreme case and the most relevant one: `qemu` translates each guest instruction, and a tight interpreter loop is pathological for that. Bulk `sha256` shows "only" 14× because the overhead amortises across large blocks. The 2 s vs 70 s container lifecycle directly explains the previously-unexplained "30–60 minutes to start 16 runners".

Cross-host reference, same benchmark:

host	imports	sha256 75 MB	interpreter loop
server3 (M2 Max, native)	0.38 s	0.03 s	0.24 s
server1 (i7-8700B, **native**)	0.57 s	0.20 s	0.65 s
server1 (i7-8700B, *emulated*)	18.21 s	2.76 s	45.95 s

A 2018 Intel part is genuinely slower than an M2 Max — ~2.7× on the loop — but that is a *hardware* difference of the same order, whereas emulation was a difference of two orders. The distinction matters for capacity planning: after the change these hosts are comparable workers; before it they were liabilities.

6.4 Capacity recovered

Two effects, one immediate and one enabled:

1. **32 runner slots** on two hosts became usable for real work rather than nominally present and effectively idle.
2. One Intel host had 34 GB of physical RAM but a **7.9 GiB Docker VM cap** — an artificial limit, not a hardware one. Raising it to 24 GiB cleared the `Requires(memory_gb=10)` floor and made it a **second heavy-tier host**.

The effect on the county fan-out, measured:

	single heavy host	two heavy hosts
counties built	2,522	2,522 (identical work)
wall clock	2.00 h	**1.77 h** (−12%)
aggregate server-time	7.31 h	11.18 h
parallelism (server-time ÷ wall)	3.7×	**6.3×**
share carried by the new host	—	**42%**

Both runs built exactly the same 2,522 extracts, which makes this a clean comparison — but the result is more interesting than a simple speed-up, and the two directions of change need explaining together.

Parallelism rose from 3.7× to 6.3×: the `width: 6` concurrency cap is now actually achieved rather than throttled by one host's memory ceiling. Aggregate server-time *also* rose, from 7.31 h to 11.18 h, for identical output — because the hosts that joined are slower, so the same regions cost more machine-time on them. Wall clock fell only 12% despite parallelism nearly doubling, for the same reason.

The general form of the result: **adding slower capacity to a bounded fan-out buys sub-linear wall-clock improvement and costs super-linear aggregate machine-time.** It is still worth doing here — the added hosts were otherwise idle, so their machine-time is free — but a fleet billed per core-hour would find this trade unattractive, and a report showing only wall clock would hide it.

6.5 An own-goal worth documenting: registry GC

Making the image multi-architecture exposed a self-inflicted failure that deserves recording, because it is a general hazard of container registries.

Reclaiming disk earlier in the campaign, we ran `registry garbage-collect` against a **live** registry — which the documentation forbids, requiring the registry be stopped or read-only — and did not restart it afterwards. GC deleted blob data from disk while the running process continued answering `HEAD → 200` from its `blobdescriptor: inmemory` cache. `buildx`'s pre-push existence check was therefore told the layers already existed, skipped uploading them, and pushed a manifest referencing blobs that were gone.

The result was an image that passed every structural check and could not be pulled by anything, including the registry's own host:

```
short read: expected 111686382 bytes but got 0: unexpected EOF
```

Twelve of twenty-four `amd64` blobs had no data. `HEAD` said 200 for all of them. The fix was to restart the registry (clearing the stale cache) and rebuild; the durable fix was the ranged-`GET` verification of §5/R2.

---

7. Prescriptions

P1. Grep the primitive, not the handler. Every missing-guarantee defect in this campaign was findable in seconds by searching for the shared blocking call and asking whether all call sites were guarded:

```
grep -B3 "bootstrap_batched(" handlers.py | grep -c "_heartbeating"   # 1 of 2 → bug
```

Waiting for the watchdog to reveal them cost hours each.

P2. Recovery paths must inherit or refuse, never improvise. Where a recovery path lacks the context to derive a guarantee, inheriting it from the prior artefact is both simpler and more correct than re-deriving. Where inheritance is impossible, warn — do not emit a silently unconstrained unit of work.

P3. Separate "did nothing" from "did everything" in every report. Report built *and* skipped. Name the knob responsible when the answer is zero.

P4. Verify data, not metadata. `HEAD` is not evidence of a readable blob; "platform listed" is not evidence of a pullable image; "task completed" is not evidence of a published artifact. Verify the property you depend on.

P5. Assert impossibility. Values that cannot physically be true (usage above capacity; a peak exceeding a VM's size) should be flagged by the tool that computes them. This costs one comparison and catches a class of attribution bugs that no test enumerates.

P6. Treat architecture homogeneity as an assumption to be checked. It is not visible in code review, does not fail, and presents as ordinary slowness.

---

8. Threats to validity and scope

Sample size. This is one fleet, one domain, and one three-day campaign. The defect *shape* (§3) recurs nine times within it, which we consider suggestive rather than demonstrated; we have not surveyed other systems for it.

Benchmark scope. The §6.3 measurements are Python-level workloads (interpreter startup, imports, hashing, a tight loop) chosen because they mirror what a runner actually does between tasks. They are not a general CPU benchmark, and the 71× figure in particular is the *most* favourable case for the change; the sha256 result (14×) is closer to what compute-bound native code will see.

The fan-out comparison is not controlled (§6.4), as stated there.

Attribution. Per-host time and log attribution rely on claim-history log lines. Tasks predating that logging, or emitted before any claim, are reported as unattributed rather than assigned — 1 second in the runs analysed here, but a larger figure would weaken the per-host conclusions.

A structural limit we did not solve. OSM `admin_level=6` relations yield **2,522 of the 3,167** county extracts in the bucket. The remaining 645 — verified per-state (alabama 57 of 67, texas 227 of 257, georgia 143 of 159, virginia 118 of 129) — originate from the earlier Census TIGER build and **cannot be regenerated by the OSM-boundary path at all**. They will age indefinitely. This is a 20.4% permanent staleness floor in the county tier, it is currently invisible in every report, and by the argument of §5 that invisibility is the more serious half of the problem.

---

9. Conclusion

The pipeline described in the companion paper was correct in the sense that matters least: its happy path worked. Completing the extraction required fixing nine defects, none of which were in that path, and four of which reported success while doing nothing at all.

The unifying property is that **guarantees are established per call site, not per system.** A resource floor, a liveness signal, a scoping predicate, a freshness rule — each is real only where it was written. The primary path carried all of them; recovery paths, written later and with less context, carried none. Systems that recover well are therefore systems where recovery paths are held to the same invariants as primary ones, and the cheapest way we have found to enforce that is mechanical: search for the primitive and count the guards.

The second result is smaller in scope and larger in leverage. Two of four hosts had been running every workload under CPU emulation, at a measured cost of one to two orders of magnitude, and the fix required no code changes and four minutes. It went undone for months because emulation is slow rather than broken, and slowness has no error condition. That is the same reason a 36-day-old data tier survived beside a same-day one: **the failure modes that persist are the ones that do not announce themselves**, and the engineering discipline that matters most is making systems say plainly when they have done nothing.